

1 Bitwise Trick

1.1 tricks

```

1 bool isOdd(int x) {
2     return x & 1;
3 }
4
5 int isolateRightmostSetBit(int x) {
6     return x & -x; // 取得最右邊的 1
7 }
8
9 bool isPowerOfTwo(int x) {
10    return x > 0 && (x & (x - 1)) == 0;
11 }
12
13 int countSetBits(int x) {
14     int count = 0;
15     while (x) {
16         x &= (x - 1); // 清除最右邊的 1
17         count++;
18     }
19     return count;
20 }
21
22 int turnOffRightmostSetBit(int x) {
23     return x & (x - 1); // 將最右邊的 1 設為 0
24 }
25
26 int setBit(int x, int pos) {
27     return x | (1 << pos); // 設該位元為 1
28 }
29
30 int clearBit(int x, int pos) {
31     return x & ~(1 << pos); // 設該位元為 0
32 }
33
34 int toggleBit(int x, int pos) {
35     return x ^ (1 << pos); // 切換該位元
36 }
37
38 int getBit(int x, int pos) {
39     return (x >> pos) & 1; // 取得該位元值
40 }
41
42 int modPowerOfTwo(int x, int mod) {
43     return x & (mod - 1); // x % mod, mod 必須是 2 的冪次
44 }
45
46 // a + b = (a ^ b) + 2 * (a & b)
47 int add(int a, int b) {
48     while (b != 0) {

```

```

49         int carry = a & b; // 計算進位
50         a = a ^ b; // 無進位相加
51         b = carry << 1; // 將進位左移一位
52     }
53     return a;
54 }
55 // a - b = (a ^ b) - 2 * ((~a) & b)
56 int subtract(int a, int b) {
57     while (b != 0) {
58         int borrow = (~a) & b; // 計算借位
59         a = a ^ b; // 無借位相減
60         b = borrow << 1; // 將借位左移一位
61     }
62     return a;
63 }
64 // a * b = sum(a << i) for each set bit i in b
65 int multiply(int a, int b) {
66     int result = 0;
67     while (b > 0) {
68         if (b & 1) { // 如果 b 的最低位是 1
69             result = add(result, a); // 使用上面的加法函數
70         }
71         a <<= 1; // 將 a 左移一位 (相當於乘以 2)
72         b >>= 1; // 將 b 右移一位 (相當於除以 2)
73     }
74     return result;
75 }
76 // a / b = sum(1 << i) for each set bit i in a
77 int divide(int a, int b) {
78     if (b == 0) throw std::invalid_argument("Division by zero");
79     int result = 0;
80     int power = 1;
81     int tempB = b;
82     while (tempB <= a && tempB > 0) {
83         // 防止溢位
84         tempB <<= 1;
85         power <<= 1;
86     }
87     while (power > 1) {
88         tempB >>= 1;
89         power >>= 1;
90         if (a >= tempB) {
91             a = subtract(a, tempB);
92             // 使用上面的減法函數

```

```

93         result = add(result, power); // 使用上面的加法函數
94     }
95     }
96     return result;
97 }
98
99 int findUnique(const std::vector<int>& nums) {
100    int unique = 0;
101    for (int num : nums) {
102        unique ^= num; // XOR 相同的數字會抵消
103    }
104    return unique;
105 }
106
107 int graycode(int n) { // n 轉 Gray code
108     return n ^ (n >> 1);
109 }

```

2 DP

2.1 LIS

```

1 ll LISLength(const vector<ll>& arr) {
2     vector<ll> d;
3     for (ll x : arr) {
4         auto it = lower_bound(d.begin(), d.end(), x);
5         if (it == d.end()) d.push_back(x);
6         else *it = x;
7     }
8     return d.size();
9 }
10
11 // find LIS sequence
12 for (ll i = 0; i < num.size(); ++i) {
13     if (lis.empty() || lis.back() < num[i]) {
14         lis.push_back(num[i]);
15         dp[i] = lis.size();
16     }
17     else {
18         auto iter = lower_bound(lis.begin(), lis.end(), num[i]);
19         dp[i] = iter - lis.begin() + 1;
20         *iter = num[i];
21     }
22 }
23 ll length = lis.size();
24 for (ll i = num.size() - 1; i >= 0; --i) {
25     if (dp[i] == length) {

```

```

26         ans.push_back(num[i]);
27         length--;
28     }
29 }

```

2.2 Knapsack

```

1 // 0/1 knapsack
2
3 const ll INF = 9e18;
4
5 ll n, wmx; cin >> n >> wmx;
6 vector<ll> w(n + 1); for (ll i = 1; i <= n; ++i) cin >> w[i];
7 vector<ll> v(n + 1); for (ll i = 1; i <= n; ++i) cin >> v[i];
8
9 vector<ll> dp(wmx + 1, -INF); dp[0] = 0;
10 for (ll i = 1; i <= n; ++i) {
11     for (ll j = wmx; j >= w[i]; --j) {
12         dp[j] = max(dp[j], dp[j - w[i]] + v[i]);
13     }
14 }
15
16 ll mx = dp[0];
17 for (ll j = 1; j <= wmx; ++j) {
18     mx = max(mx, dp[j]);
19 }
20 cout << mx << '\n';
21
22 // -----
23 // Unbounded
24 // just change the inner for loop to the following line
25 // for (ll j = w[i]; j <= wmx; ++j) ...

```

2.3 Subset DP

```

1 for (ll s = 1; s < (1LL << n); ++s) {
2     for (ll sbs = s; sbs > 0; sbs = ((sbs - 1) & s)) {
3         ll c = s - sbs; // ll c = sbs ^ s; // this one is equivalent
4     }
5 }
6 // O(3^n), n <= 16
7 // sbs 是降序的 s 的子集
8 // c 是升序的 s 的子集

```

2.4 Interval DP

```

1 ll n; // the length of the array
2
3 vector<ll> a(n + 1);
4
5 vector<vector<ll>> dp(n + 1, vector<ll>(n + 1));
6
7 for (ll i = 1; i <= n; ++i) {
8     dp[i][i] = "initial value"; // length 1 interval [i, i]
9 }
10 for (ll len = 2; len <= n; ++len) {
11     // loop through all length from 2 to n
12     for (ll l = 1; l <= n - len + 1; ++l) {
13         ll r = l + len - 1;
14         dp[l][r] = ...; // interval [l, r]
15     }
16 }
17 dp[1][n] // the answer (interval [1, n])

```

2.5 LCS

```

1 ll n; cin >> n;
2 string s; cin >> s; s = " " + s;
3 ll m; cin >> m;
4 string t; cin >> t; t = " " + t;
5
6 vector<vector<ll>> dp(n + 1, vector<ll>(m + 1, 0));
7 for (ll i = 1; i <= n; ++i) {
8     for (ll j = 1; j <= m; ++j) {
9         if (s[i] == t[j]) {
10             dp[i][j] = dp[i - 1][j - 1] + 1;
11         }
12         else {
13             dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
14         }
15     }
16 }
17 cout << dp[n][m] << '\n'; // the length of LCS

```

2.6 Digit DP

```

1 string num;
2
3 ll dp[the max len of num + 1][2][2][...];

```

```

4 memset(dp, -1, sizeof(dp));
5
6 ll count(ll pos, bool tight, bool
  leadingZero, ...) {
7     if (dp[pos][tight][leadingZero
  ][...] != -1) {
8         return dp[pos][tight][
  leadingZero][...];
9     }
10
11     if (base-case) { // e.g. pos
  == num.size() or other
  base case condition
  // do something
12     }
13
14     ll res = 0;
15     ll up = (tight ? num[pos] - '0'
  : 9);
16     for (ll d = 0; d <= up; ++d) {
17         res += count(
18             pos + 1,
19             tight and (d == num[pos]
  - '0'),
20             leadingZero and (d ==
  0),
21             ...
22         );
23     }
24
25     return dp[pos][tight][
  leadingZero][...] = res;
26 }
27
28 count(0, true, true, ...) // the
  answer
29
30 // =====
31 // =====
32
33 // AtCoder ABC154E
34 // Find the number of integers
  between 1 and N (inclusive)
35 // that contains exactly K non-zero
  digits when written in base
  ten.
36 // 1 <= N <= 10^100, 1 <= K <= 3
37
38 #include <bits/stdc++.h>
39 using namespace std;
40 using ll = long long;
41
42 ll k;
43 string num;
44
45 ll dp[102][2][4];
46
47 ll count(ll pos, bool tight, ll cnt
  ) {
48     if (dp[pos][tight][cnt] != -1)
49     {
50         return dp[pos][tight][cnt];
51     }
52
53     if (cnt == 0) return dp[pos][
  tight][cnt] = 1;

```

```

54     else if (pos == num.size())
55     {
56         return dp[pos][tight][cnt]
  = 0;
57     }
58
59     ll res = 0;
60     ll up = (tight ? num[pos] - '0'
  : 9);
61     for (ll d = 0; d <= up; ++d) {
62         res += count(
63             pos + 1,
64             tight and (d == num[pos]
  - '0'),
65             cnt + (d == 0 ? 0 : -1)
66         );
67     }
68
69     return dp[pos][tight][cnt] =
  res;
70 }
71
72 int main() {
73     ios::sync_with_stdio(false);
74     cin.tie(nullptr);
75
76     cin >> num >> k;
77
78     memset(dp, -1, sizeof(dp));
79
80     cout << count(0, true, k) << '\
  n';
81
82     return 0;
83 }

```

3 D&C

3.1 MergeSort Finds the Number of Inversions

```

1 void merge(vector<int>& arr, ll l,
  ll r, ll x, ll y) {
2     ll left = l;
3     vector<ll> tmp; tmp.reserve(y - l
  + 1);
4     while (l <= r and x <= y) {
5         if (arr[l] <= arr[x]) {
6             tmp.push_back(arr[l++]);
7         }
8         else {
9             tmp.push_back(arr[x++]);
10        }
11    }
12    while (l <= r) {
13        tmp.push_back(arr[l++]);
14    }
15    while (x <= y) {
16        tmp.push_back(arr[x++]);
17    }
18    for (ll i = left; i <= y; ++i) {
19        arr[i] = tmp[i - left];

```

```

20    }
21 }
22 ll mergeSort(vector<ll>& arr, ll l,
  ll r) {
23     if (l == r) return 0;
24     ll mid = l + (r - l)/2;
25     ll lcnt = mergeSort(arr, l, mid
  );
26     ll rcnt = mergeSort(arr, mid +
  1, r);
27
28     // ----- main Logic
29     // -----
30     ll cnt = lcnt + rcnt;
31     ll a = l, b = mid, c = mid + 1,
  d = r; // c is the
  current checking position
32     while (a <= b) {
33         while (c <= d and arr[a] >
  arr[c]) c += 1;
34         cnt += c - (mid + 1);
35         a += 1;
36     }
37
38     // -----
39     merge(arr, l, mid, mid + 1, r);
40
41     return cnt;
42 }

```

4 Data Structure

4.1 Segment Tree

```

1 // CSES Range Updates and Sums
2 // 1. Increase each value in range
  [a,b] by x.
3 // 2. Set each value in range [a,b]
  to x.
4 // 3. Calculate the sum of values
  in range [a,b].
5
6 #include <bits/stdc++.h>
7 using namespace std;
8 using ll = long long;
9
10 #define lson 2*n + 1
11 #define rson 2*n + 2
12
13 class Node {
14 public:
15     ll val;
16     ll setVal;
17     ll addVal;
18     bool isSet;
19 };
20
21 ll sz, q;
22 vector<ll> a;

```

```

23 vector<Node> nds;
24
25 void build(ll l, ll r, ll n = 0) {
26     if (l == r) {
27         nds[n].val = a[l];
28         nds[n].setVal = 0;
29         nds[n].addVal = 0;
30         nds[n].isSet = false;
31         return;
32     }
33     ll mid = l + (r - l)/2;
34     build(l, mid, lson);
35     build(mid + 1, r, rson);
36     nds[n].val = nds[lson].val +
  nds[rson].val;
37 }
38
39 void push(ll l, ll r, ll n) {
40     ll mid = l + (r - l)/2;
41
42     if (nds[n].isSet) {
43         nds[lson].val = nds[n].
  setVal*(mid - l + 1);
44         nds[lson].setVal = nds[n].
  setVal;
45         nds[lson].addVal = 0;
46         nds[lson].isSet = true;
47
48         nds[rson].val = nds[n].
  setVal*(r - (mid + 1)
  + 1);
49         nds[rson].setVal = nds[n].
  setVal;
50         nds[rson].addVal = 0;
51         nds[rson].isSet = true;
52
53         nds[n].isSet = false;
54     }
55
56     nds[lson].val += nds[n].addVal
  *(mid - l + 1);
57     nds[lson].addVal += nds[n].
  addVal;
58
59     nds[rson].val += nds[n].addVal
  *(r - (mid + 1) + 1);
60     nds[rson].addVal += nds[n].
  addVal;
61
62     nds[n].addVal = 0;
63 }
64
65 void setVal(ll x, ll y, ll val, ll
  l, ll r, ll n = 0) {
66     if (l == x and r == y) {
67         nds[n].val = val*(y - x +
  1);
68         nds[n].setVal = val;
69         nds[n].addVal = 0;
70         nds[n].isSet = true;
71         return;
72     }
73
74     push(l, r, n);
75     ll mid = l + (r - l)/2;
76     if (y <= mid) {

```

```

77         setVal(x, y, val, l, mid,
  lson);
78     } else if (x >= mid + 1) {
79         setVal(x, y, val, mid + 1,
  r, rson);
80     } else {
81         setVal(x, mid, val, l, mid,
  lson);
82         setVal(mid + 1, y, val, mid
  + 1, r, rson);
83     }
84     nds[n].val = nds[lson].val +
  nds[rson].val;
85 }
86
87 void addVal(ll x, ll y, ll val, ll
  l, ll r, ll n = 0) {
88     if (l == x and r == y) {
89         nds[n].val += val*(y - x +
  1);
90         nds[n].addVal += val;
91         return;
92     }
93     push(l, r, n);
94     ll mid = l + (r - l)/2;
95     if (y <= mid) {
96         addVal(x, y, val, l, mid,
  lson);
97     } else if (x >= mid + 1) {
98         addVal(x, y, val, mid + 1,
  r, rson);
99     } else {
100         addVal(x, mid, val, l, mid,
  lson);
101         addVal(mid + 1, y, val, mid
  + 1, r, rson);
102     }
103     nds[n].val = nds[lson].val +
  nds[rson].val;
104 }
105
106 ll query(ll x, ll y, ll l, ll r, ll
  n = 0) {
107     if (l == x and r == y) return
  nds[n].val;
108     push(l, r, n);
109     ll mid = l + (r - l)/2;
110     if (y <= mid) {
111         return query(x, y, l, mid,
  lson);
112     } else if (x >= mid + 1) {
113         return query(x, y, mid + 1,
  r, rson);
114     } else {
115         ll a = query(x, mid, l, mid,
  lson);
116         ll b = query(mid + 1, y,
  mid + 1, r, rson);
117         return a + b;
118     }
119 }
120
121 int main() {
122     ios::sync_with_stdio(false);
123     cin.tie(nullptr);
124 }

```

4.3 Treap

```

125 cin >> sz >> q;
126
127 a.resize(sz + 1);
128 for (ll i = 1; i <= sz; ++i)
129     cin >> a[i];
130
131 nds.resize(4*sz);
132 build(1, sz);
133
134 while (q--) {
135     ll act; cin >> act;
136     if (act == 1) {
137         ll l, r, val; cin >> l
138         >> r >> val;
139         addVal(1, r, val, 1, sz
140             );
141     } else if (act == 2) {
142         ll l, r, val; cin >> l
143         >> r >> val;
144         setVal(1, r, val, 1, sz
145             );
146     } else if (act == 3) {
147         ll l, r; cin >> l >> r;
148         cout << query(1, r, 1,
149             sz) << '\n';
150     }
151 }
152
153 return 0;
154 }
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

```

223     root = merge(root, new Node 275
        (s[i]));
224 }
225
226 rep (q) {
227     ll l, r;
228     cin >> l >> r;
229     rev_range(root, l, r);
230 }
231
232 string res = "";
233 inorder(root, res);
234 cout << res << '\n';
235
236 return 0;
237 }
238 //
239 //
240 //
241 // CSES Reversals and Sums
242 // Given an array of n integers,
243 // you have to process following
244 // operations:
245 // reverse a subarray
246 // calculate the sum of values in a
247 // subarray
248
249 #include <bits/stdc++.h>
250 using namespace std;
251 using ll = long long;
252 using pll = pair<ll, ll>;
253 template<class T> using vec =
254     vector<T>;
255 #define endl '\n'
256 #define ff first
257 #define ss second
258 #define pb push_back
259 #define rep(n) for (ll _ = 1; _ <=
260     n; ++_)
261
262 mt19937 rng(chrono::steady_clock::
263     now().time_since_epoch().count
264     ());
265
266 class Node {
267 public:
268     Node* l, * r;
269     ll val, pri, sz;
270     ll sum;
271     bool rev_tag;
272     Node(ll _val) {
273         l = r = nullptr;
274         val = _val;
275         pri = rng();
276         sz = 1;
277         sum = _val;
278         rev_tag = false;
279     }
280
281     void pull() {
282         sz = 1;
283         if (l) sz += l->sz;
284         if (r) sz += r->sz;
285
286         sum = val;
287         if (l) sum += l->sum;
288         if (r) sum += r->sum;
289     }
290
291     void push() {
292         if (rev_tag) {
293             swap(l, r);
294             if (l) l->rev_tag = !l
295                 ->rev_tag;
296             if (r) r->rev_tag = !r
297                 ->rev_tag;
298             rev_tag = false;
299         }
300     }
301
302     Node* merge(Node* a, Node* b) {
303         if (!a || !b) return a ? a : b;
304         if (a->pri < b->pri) {
305             a->push();
306             a->r = merge(a->r, b);
307             a->pull();
308             return a;
309         } else {
310             b->push();
311             b->l = merge(a, b->l);
312             b->pull();
313             return b;
314         }
315     }
316
317     void splsz(Node* root, Node* a,
318         Node* b, ll s) {
319         if (!root) {
320             a = b = nullptr;
321             return;
322         }
323         root->push();
324         ll lsr = root->l ? root->l->sz
325             : 0;
326         if (lsr + 1 <= s) {
327             a = root;
328             splsz(root->r, a->r, b, s -
329                 (lsr + 1));
330             a->pull();
331         } else {
332             b = root;
333             splsz(root->l, a, b->l, s);
334             b->pull();
335         }
336     }
337
338     void rev_range(Node* root, ll l,
339         ll r) { // 1-indexed
340         Node* left, * mid, * right;
341         splsz(root, left, mid, l - 1);
342         splsz(mid, mid, right, r - l +
343             1);
344         if (mid) mid->rev_tag = !mid->
345             rev_tag;
346     }
347
348     int main() {
349         ios::sync_with_stdio(false);
350         cin.tie(nullptr);
351
352         ll n, q;
353         cin >> n >> q;
354
355         Node* root = nullptr;
356         for (ll i = 0; i < n; ++i) {
357             ll val; cin >> val;
358             root = merge(root, new Node
359                 (val));
360         }
361
362         rep (q) {
363             ll act, l, r;
364             cin >> act >> l >> r;
365             if (act == 1) {
366                 rev_range(root, l, r);
367             } else if (act == 2) {
368                 cout << qry_range_sum(
369                     root, l, r) <<
370                     endl;
371             }
372         }
373         return 0;
374     }
375
376 // SPOJ ORDERSET
377 // In this problem, you have to
378 // maintain a dynamic set of
379 // numbers
380 // which support the two
381 // fundamental operations
382 // INSERT(S,x): if x is not in
383 // S, insert x into S
384 // DELETE(S,x): if x is in S,
385 // delete x from S
386 // and the two type of queries
387 // K-TH(S) : return the k-th
388 // smallest element of S
389
390 root = merge(left, mid);
391 root = merge(root, right);
392 }
393
394 ll qry_range_sum(Node* root, ll l,
395     ll r) {
396     Node* left, * mid, * right;
397     splsz(root, left, mid, l - 1);
398     splsz(mid, mid, right, r - l +
399         1);
400     ll res = 0;
401     if (mid) res = mid->sum;
402     root = merge(left, mid);
403     root = merge(root, right);
404     return res;
405 }
406
407 int main() {
408     ios::sync_with_stdio(false);
409     cin.tie(nullptr);
410
411     ll n, q;
412     cin >> n >> q;
413
414     Node* root = nullptr;
415     for (ll i = 0; i < n; ++i) {
416         ll val; cin >> val;
417         root = merge(root, new Node
418             (val));
419     }
420
421     rep (q) {
422         ll act, l, r;
423         cin >> act >> l >> r;
424         if (act == 1) {
425             rev_range(root, l, r);
426         } else if (act == 2) {
427             cout << qry_range_sum(
428                 root, l, r) <<
429                 endl;
430         }
431     }
432     return 0;
433 }
434
435 // COUNT(S,x): return the
436 // number of elements of S
437 // smaller than x
438
439 #include <bits/stdc++.h>
440 using namespace std;
441 using ll = long long;
442 using pll = pair<ll, ll>;
443 template<class T> using vec =
444     vector<T>;
445 #define endl '\n'
446 #define ff first
447 #define ss second
448 #define pb push_back
449 #define rep(n) for (ll _ = 1; _ <=
450     n; ++_)
451
452 mt19937 rng(chrono::steady_clock::
453     now().time_since_epoch().count
454     ());
455
456 class Node {
457 public:
458     Node* l, * r;
459     ll val, pri, sz;
460     Node(ll _val) {
461         l = r = nullptr;
462         val = _val;
463         pri = rng();
464         sz = 1;
465     }
466
467     void pull() {
468         sz = 1;
469         if (l) sz += l->sz;
470         if (r) sz += r->sz;
471     }
472
473     Node* merge(Node* a, Node* b) {
474         if (!a || !b) return a ? a : b;
475         if (a->pri < b->pri) {
476             a->r = merge(a->r, b);
477             a->pull();
478             return a;
479         } else {
480             b->l = merge(a, b->l);
481             b->pull();
482             return b;
483         }
484     }
485
486     void split(Node* root, Node* a,
487         Node* b, ll k) {
488         if (!root) {
489             a = b = nullptr;
490             return;
491         }
492         if (root->val <= k) {
493             a = root;
494             split(root->r, a->r, b, k);
495             a->pull();
496         } else {
497             b = root;
498             split(root, left, right, k - 1);
499             ll res = 0;
500             if (left) res = left->sz;
501             root = merge(left, right);
502             return res;
503         }
504     }
505
506     ll kth_smallest(Node* root, ll k) {
507         assert(root && 1 <= k && k <=
508             root->sz);
509         Node* left, * mid, * right;
510         splsz(root, left, mid, k - 1);
511         splsz(mid, mid, right, 1);
512         ll res = mid->val;
513         root = merge(left, mid);
514         root = merge(root, right);
515         return res;
516     }
517
518     void insert(Node* root, ll k) {
519         Node* left, * right;
520         split(root, left, right, k);
521         Node* a = new Node(k);
522         root = merge(left, a);
523     }
524 }

```

```

500     root = merge(root, right);
501 }
502
503 void remove_multi(Node*& root, ll k
504 ) {
505     Node* left,* mid,* right;
506     split(root, left, mid, k - 1);
507     split(mid, mid, right, k);
508     root = merge(left, right);
509 }
510
511 void remove_single(Node*& root, ll
512 k) {
513     Node* left,* mid,* right;
514     split(root, left, mid, k - 1);
515     splsz(mid, mid, right, 1);
516     if (!mid || mid->val != k) {
517         root = merge(left, mid);
518         root = merge(root, right);
519     } else {
520         root = merge(left, right);
521     }
522 }
523
524 int main() {
525     ios::sync_with_stdio(false);
526     cin.tie(nullptr);
527
528     // The element can be
529     // duplicated in the treap
530     // of the implementation of
531     // this program
532
533     // But the element cannot be
534     // duplicated in this problem
535     // (Use "count" to check
536     // whether the element exists
537     // )
538
539     ll q; cin >> q;
540
541     Node* root = nullptr;
542
543     rep (q) {
544         char act; cin >> act;
545         ll k; cin >> k;
546         if (act == 'I') {
547             if (count(root, k) ==
548                 0) {
549                 insert(root, k);
550             }
551             } else if (act == 'D') {
552                 remove_single(root, k);
553             } else if (act == 'K') {
554                 if (!root || k > root->
555                     sz) {
556                     cout << "invalid"
557                         << endl;
558                 } else {
559                     cout <<
560                         kth_smallest(
561                             root, k) <<
562                         endl;
563                 }
564             } else if (act == 'C') {
565

```

4.4 Sparse Table

```

1 // if the max size of arr is 200000
2 vector<ll> arr;
3
4 // -----
5 // Sparse Table
6 const ll lgmx = 17; // floor(log2
7 (200000))
8 ll rmq[lgmx + 1][200000 + 1];
9 void init(ll n) { // O(n log n)
10     for (ll i = 1; i <= n; ++i) rmq
11         [0][i] = arr[i];
12     for (ll h = 1; h <= lgmx; ++h)
13         for (ll i = 1; i + (1 << h)
14             - 1 <= n; ++i)
15             rmq[h][i] = min(rmq[h -
16                 1][i], rmq[h -
17                     1][i + (1 << h -
18                         1)]);
19 }
20 ll flg ull x) {return 63 -
21     __builtin_clzll(x);}
22 ll query(ll l, ll r) { // O(1)
23     ll h = flg(r - l + 1);
24     return min(rmq[h][l], rmq[h][r
25         - (1 << h) + 1]);
26 }
27 // -----
28 // initialize the array
29 ll n; cin >> n;
30 arr.resize(n + 1);
31 for (ll i = 1; i <= n; ++i) cin >>
32     arr[i];
33 // initialize the sparse table
34 init(n);

```

4.5 DSU

```

1 ll cc;
2 vector<ll> djs, sz;
3 ll find(ll u) {
4     if (u == djs[u]) return u;
5     return djs[u] = find(djs[u]);
6 }
7 void join(ll u, ll v) {
8     u = find(u);
9     v = find(v);

```

```

552     cout << count_lt(root,
553         k) << endl;
554 }
555 }
556 return 0;
557 }

```

```

10 if (u == v) return; // don't
11     forgot this line
12 if (sz[u] < sz[v]) swap(u, v);
13 djs[v] = u;
14 sz[u] += sz[v];
15 cc -= 1;
16 }
17 void init(ll n) {
18     djs.clear(); djs.resize(n + 1);
19     for (ll i = 1; i <= n; ++i) djs
20         [i] = i;
21     sz.clear(); sz.resize(n + 1, 1)
22     ;
23     cc = n;
24 }

```

5 Geometry

5.1 Line Segment Intersection Test

```

1 bool isSegInter(const vec& a, const
2     vec& b, const vec& c, const vec
3     & d) {
4     ll ori1 = ori(a, b, c);
5     ll ori2 = ori(a, b, d);
6     ll ori3 = ori(c, d, a);
7     ll ori4 = ori(c, d, b);
8     if (isCollinear(a, b, c) &&
9         isCollinear(a, b, d)) {
10         return isOnSeg(a, b, c) ||
11             isOnSeg(a, b, d) ||
12             isOnSeg(c, d, a) ||
13             isOnSeg(c, d, b)
14         );
15 }
16 return ori1 * ori2 <= 0 && ori3
17     * ori4 <= 0;
18 }

```

5.2 Simple Polygon Area

```

1 // pts = {p0, p1, ... pn-1, p0}, 0-
2     based
3 // for simple polygon area
4 ll shoelace2(const vector<vec> &pts
5 ) {
6     ll n = pts.size() - 2, res = 0;
7     for (ll i = 0; i <= n; ++i) {
8         res += pts[i].x * pts[i +
9             1].y;
10         res -= pts[i].y * pts[i +
11             1].x;
12     }
13     return abs(res);
14 }

```

5.3 Vector

```

1 class vec {
2 public:
3     ll x, y;
4     vec() {}
5     vec(ll _x, ll _y) : x(_x), y(_y) {}
6     vec operator+(const vec& v)
7     const {
8         return vec(this->x + v.x,
9             this->y + v.y);
10 }
11 vec operator-(const vec& v)
12 const {
13     return vec(this->x - v.x,
14         this->y - v.y);
15 }
16 operator*(const vec& v)
17 const {
18     return this->x * v.x + this
19         ->y * v.y;
20 }
21 operator^(const vec& v)
22 const {
23     return this->x * v.y - this
24         ->y * v.x;
25 }
26 bool operator<(const vec& v)
27 const {
28     if (this->x != v.x) return
29         this->x < v.x;
30     return this->y < v.y;
31 }
32 vec& operator=(const vec& v) {
33     this->x = v.x;
34     this->y = v.y;
35     return *this;
36 }
37 };
38
39 ll sign(ll x) {
40     if (x == 0) return 0;
41     if (x < 0) return -1;
42     return 1;
43 }
44
45 ll ori(const vec& o, const vec& a,
46     const vec& b) {
47     return sign((a - o) ^ (b - o));
48 }
49
50 bool isCollinear(const vec& a,
51     const vec& b, const vec& c) {
52     return ori(a, b, c) == 0;
53 }
54
55 bool isOnSeg(const vec& a, const
56     vec& b, const vec& p) {
57     return isCollinear(a, b, p) &&
58         sign((p - a) * (p - b)) <=
59             0;
60 }

```

5.4 Convex Hull

```

1 // pts = {p0, p1, ... pn-1}, 0-
2     based
3 // the points in pts should be
4     distinct
5 vector<vec> convexHull(const vector
6     <vec> &pts) {
7     vector<vec> pts = _pts;
8     sort(pts.begin(), pts.end());
9
10     ll n = pts.size();
11     vector<vec> hull(1, pts[0]);
12     for (ll i = 1; i < n; ++i) {
13         while (hull.size() >= 2 &&
14             ori(hull[hull.size()
15                 - 2], hull.
16                 back(), pts[i])
17                 < 0) {
18             hull.pop_back();
19         }
20         hull.push_back(pts[i]);
21     }
22
23     ll m = hull.size();
24     for (ll i = n - 2; i >= 0; --i)
25     {
26         while (hull.size() - m + 1
27             >= 2 &&
28             ori(hull[hull.size()
29                 - 2], hull.
30                 back(), pts[i])
31                 < 0) {
32             hull.pop_back();
33         }
34         hull.push_back(pts[i]);
35     }
36     hull.pop_back();
37     return hull;
38 }

```

6 Graph

6.1 Tarjan

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using pll = pair<ll, ll>;
5 #define pb push_back
6
7 ll n, m, t = 0, sccCnt = 0, sccIdx
8     = 0;
9 stack<ll> stk;
10 vector<bool> inStack;
11 vector<ll> in, low, scc;
12 vector<vector<ll>> adj;
13 void dfs(ll u) {
14     low[u] = in[u] = ++t;

```



```

14 stk.push(u);
15 inStack[u] = true;
16 for (ll v : adj[u]) {
17     if (in[v] == -1) {
18         dfs(v);
19         low[u] = min(low[u],
20                     low[v]);
21     } else if (inStack[v]) {
22         low[u] = min(low[u],
23                     low[v]);
24     }
25 }
26 if (in[u] == low[u]) {
27     sccCnt += 1;
28     sccIdx += 1;
29     ll cur;
30     do {
31         cur = stk.top(); stk.
32         pop();
33         inStack[cur] = false;
34         scc[cur] = sccIdx;
35     } while (cur != u);
36 }
37 int main() {
38     ios::sync_with_stdio(false);
39     cin.tie(nullptr);
40
41     cin >> n >> m;
42     inStack.resize(n + 1);
43     in.resize(n + 1, -1);
44     low.resize(n + 1);
45     scc.resize(n + 1);
46     adj.resize(n + 1);
47     for (ll i = 1; i <= m; ++i) {
48         ll u, v; cin >> u >> v;
49         adj[u].pb(v);
50     }
51
52     for (ll u = 1; u <= n; ++u) {
53         if (in[u] != -1) continue;
54         dfs(u);
55     }
56
57     cout << sccCnt << '\n';
58     for (ll u = 1; u <= n; ++u) {
59         cout << scc[u] << ' ';
60     } cout << '\n';
61
62     return 0;

```

6.2 Bridge

```

1 void dfs(ll u, ll pa = -1) {
2     low[u] = in[u] = ++t;
3     for (ll v : adj[u]) {
4         if (v == pa) continue;
5         if (in[v] == -1) {
6             dfs(v, u);
7             low[u] = min(low[u],
8                         low[v]);

```

```

8         if (low[v] > in[u]) "
9             edge (u, v) is
10             bridge" // find
11             bridge
12         } else if (in[v] < in[u]) {
13             low[u] = min(low[u], in
14                         [v]);
15         }
16     }
17 }

```

6.3 Bellman Ford Detects Negative Cycle

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using pll = pair<ll, ll>;
5 #define ff first
6 #define ss second
7 #define pb push_back
8 #define rep(n) for (ll _ = 1; _ <=
9     n; ++_)
10 int main() {
11     ios::sync_with_stdio(false);
12     cin.tie(nullptr);
13
14     const ll INF = 5e12 + 1000;
15
16     ll n, m; cin >> n >> m;
17     vector<ll> dis(n + 1, INF);
18     vector<vector<pll>> adj(n + 1);
19     rep (m) {
20         ll u, v, w; cin >> u >> v >> w;
21         adj[u].pb(pll(w, v));
22     }
23     for (ll i = 1; i <= n; ++i) {
24         adj[0].pb(pll(0, i));
25     }
26
27     dis[0] = 0;
28     rep (n - 1) {
29         bool updated = false;
30         for (ll u = 0; u <= n; ++u) {
31             for (const auto& [w, v] : adj
32                 [u]) {
33                 if (dis[u] < INF and dis[u]
34                     + w < dis[v]) {
35                     dis[v] = dis[u] + w;
36                     updated = true;
37                 }
38             }
39         }
40         if (not updated) break;
41     }
42
43     bool hasNegativeCycle = false;
44     for (ll u = 0; not
45         hasNegativeCycle and u <= n;
46         ++u) {

```

```

43     for (const auto& [w, v] : adj[u]
44         ]) {
45         if (dis[u] < INF and dis[u] +
46             w < dis[v]) {
47             hasNegativeCycle = true;
48             break;
49         }
50     }
51     return 0;
52 }

```

6.4 Topo-sort Kahn

```

1 int main() {
2     ios::sync_with_stdio(false);
3     cin.tie(nullptr);
4
5     ll n, m; cin >> n >> m;
6     vector<ll> indeg(n + 1, 0);
7     vector<vector<ll>> adj(n + 1);
8     rep (m) {
9         ll u, v; cin >> u >> v;
10        adj[u].pb(v);
11        indeg[v] += 1;
12    }
13
14    vector<ll> res; res.reserve(n);
15    queue<ll> q; // you can use any
16    // pool data structure here
17    for (ll u = 1; u <= n; ++u) {
18        if (indeg[u] == 0) {
19            q.push(u);
20            res.pb(u);
21        }
22    }
23    while (not q.empty()) {
24        ll u = q.front(); q.pop();
25        for (ll v : adj[u]) {
26            indeg[v] -= 1;
27            if (indeg[v] == 0) {
28                q.push(v);
29                res.pb(v);
30            }
31        }
32    }
33
34    // if (res.size() < n) // there
35    // exists a directed cycle
36
37    return 0;

```

6.5 AP

```

1 void dfs(ll u, ll pa = -1) {
2     ll ch = 0; // 紀錄子節點樹
3     low[u] = in[u] = ++t;

```

```

4     for (ll v : adj[u]) {
5         if (v == pa) continue;
6         if (in[v] == -1) {
7             ch += 1; // 子節點數加
8             dfs(v, u);
9             low[u] = min(low[u],
10                        low[v]);
11             if (pa != -1 and low[v]
12                 >= in[u]) "u is
13                 AP" // find AP
14             } else if (in[v] < in[u]) {
15                 low[u] = min(low[u], in
16                     [v]);
17             }
18         }
19     }
20     if (pa == -1 and ch >= 2) "u is
21     AP" // root 額外判斷
22 }

```

6.6 Dijkstra

```

1 int main() {
2     ios::sync_with_stdio(false);
3     cin.tie(nullptr);
4
5     const ll INF = 1e18;
6
7     ll n, m; cin >> n >> m;
8     vector<vector<pll>> adj(n + 1);
9     rep (m) {
10        ll u, v, w; cin >> u >> v >> w;
11        adj[u].pb(pll(w, v));
12    }
13
14    vector<ll> dis(n + 1, INF);
15    priority_queue<pll, vector<pll>,
16        greater<pll>> pq;
17
18    dis[1] = 0;
19    pq.push(pll(0, 1));
20    while (not pq.empty()) {
21        auto [uw, u] = pq.top(); pq.pop
22        ();
23
24        if (uw > dis[u]) continue;
25
26        for (auto [w, v] : adj[u]) {
27            if (dis[u] + w < dis[v]) {
28                dis[v] = dis[u] + w;
29                pq.push(pll(dis[v], v));
30            }
31        }
32    }
33 }

```

6.7 Contracting Vertices Based on Specific Conditions

```

1 // define a DSU first
2
3 ll v, e; cin >> v >> e; // the
4 // number of vertices and edges
5 vector<pll> edges;
6 rep (e) {
7     ll x, y; cin >> x >> y; //
8     // bidirectional edge (x, y)
9     if (fit_some_condition) join(x, y
10     );
11     else edges.pb(pll(x, y));
12 }
13
14 vector<vector<ll>> adj; //
15 // undirected graph
16 for (const pll& edge : edges) {
17     ll x = find_root(edge.ff), y =
18     find_root(edge.ss);
19     adj[x].pb(y);
20     adj[y].pb(x);
21 }

```

6.8 MST Prim

```

1 int main() {
2     ios::sync_with_stdio(false);
3     cin.tie(nullptr);
4
5     const ll INF = 1e18;
6
7     ll n, m; cin >> n >> m;
8     vector<vector<pll>> adj(n + 1);
9     rep (m) {
10        ll u, v, w; cin >> u >> v >> w;
11        adj[u].pb(pll(w, v));
12        adj[v].pb(pll(w, u));
13    }
14
15    priority_queue<pll, vector<pll>,
16        greater<pll>> pq;
17    vector<bool> vis(n + 1, false);
18    vector<ll> curw(n + 1, INF);
19
20    pq.push(pll(0, 1));
21    curw[1] = 0;
22
23    ll mstCost = 0, mstEdgesCnt =
24    0;
25    while (mstEdgesCnt < n - 1) {
26        if (pq.empty()) {
27            // the graph is
28            // disconnected (MST
29            // D.N.E.)
30            return 0;
31        }
32    }

```

```

27     }
28
29     auto [uw, u] = pq.top(); pq
        .pop();
30
31     if (uw > curw[u]) continue;
32
33     vis[u] = true;
34     mstCost += uw;
35     mstEdgesCnt += (u == 1 ? 0
        : 1);
36
37     for (const auto& [w, v] :
        adj[u]) {
38         if (not vis[v] and w <
            curw[v]) {
39             pq.push(pll(w, v));
40             curw[v] = w;
41         }
42     }
43 }
44
45 return 0;
46 }

```

6.9 Dinic

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 #define pb push_back
5 constexpr ll INF = 1e18;
6
7 struct Node {
8     ll nd, pvIdx, cap;
9     Node(ll _nd, ll _pvIdx, ll _cap
        ) : nd(_nd), pvIdx(_pvIdx)
        , cap(_cap) {}
10 };
11
12 ll n, m, st, en;
13 vector<ll> level, iter;
14 vector<vector<Node>> adj;
15
16 void bfs() {
17     level.clear(); level.resize(n +
        1, -1);
18     level[st] = 0;
19
20     queue<ll> q; q.push(st);
21     while (not q.empty()) {
22         ll sz = q.size();
23         for (ll _ = 1; _ <= sz; ++_
            ) {
24             ll u = q.front(); q.pop
                ();
25             for (const Node& v :
                adj[u]) {
26                 if (v.cap > 0 and
                    level[v.nd] ==
                    -1) {
27                     level[v.nd] =
                        level[u] +

```

```

1;
2     q.push(v.nd);
3 }
4 }
5 }
6 }
7 }
8 }
9 }
10 }
11 }
12 }
13 }
14 }
15 }
16 }
17 }
18 }
19 }
20 }
21 }
22 }
23 }
24 }
25 }
26 }
27 }
28 }
29 }
30 }
31 }
32 }
33 }
34 }
35 }
36 }
37 }
38 }
39 }
40 }
41 }
42 }
43 }
44 }
45 }
46 }
47 }
48 }
49 }
50 }
51 }
52 }
53 }
54 }
55 }
56 }
57 }
58 }
59 }
60 }
61 }
62 }
63 }
64 }
65 }
66 }
67 }
68 }
69 }
70 }
71 }
72 }
73 }
74 }
75 }
76 }
77 }
78 }
79 }
80 }
81 }
82 }

```

6.10 Tarjan 2-SAT

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using pll = pair<ll, ll>;
5 #define pb push_back
6
7 ll n, t = 0, sccCnt = 0, sccIdx =
    0;
8 stack<ll> stk;
9 vector<bool> inStack;
10 vector<ll> in, low, scc;
11 vector<vector<ll>> adj;
12 void dfs(ll u) {
13     low[u] = in[u] = ++t;
14     stk.push(u);
15     inStack[u] = true;
16     for (ll v : adj[u]) {
17         if (in[v] == -1) {
18             dfs(v);
19             low[u] = min(low[u],
                low[v]);
20         } else if (inStack[v]) {
21             low[u] = min(low[u],
                low[v]);
22         }
23     }
24     if (in[u] == low[u]) {
25         sccCnt += 1;
26         sccIdx += 1;
27         ll cur;
28         do {
29             cur = stk.top(); stk.
                pop();
30             inStack[cur] = false;
31             scc[cur] = sccIdx;
32         } while (cur != u);
33     }
34 }
35
36 int main() {
37     ios::sync_with_stdio(false);
38     cin.tie(nullptr);
39
40     ll x, y; cin >> x >> y;
41     n = 2*y;
42     inStack.resize(n + 1);
43     in.resize(n + 1, -1);
44     low.resize(n + 1);
45     scc.resize(n + 1);
46     adj.resize(n + 1);
47     for (ll i = 1; i <= x; ++i) {
48         char c1, c2;
49         ll a, b;
50         cin >> c1 >> a >> c2 >> b;
51
52         if (c1 == '+' ) {
53             adj[a + y].pb((c2 == '+'
                ? b : b + y));
54         } else {
55             adj[a].pb((c2 == '+' ?
                b : b + y));
56         }
57     }

```

```

58     if (c2 == '+' ) {
59         adj[b + y].pb((c1 == '+'
            ? a : a + y));
60     } else {
61         adj[b].pb((c1 == '+' ?
            a : a + y));
62     }
63 }
64
65 for (ll u = 1; u <= n; ++u) {
66     if (in[u] != -1) continue;
67     dfs(u);
68 }
69
70 bool valid = true;
71 string s = "";
72 for (ll u = 1; u <= y; ++u) {
73     if (scc[u] == scc[u + y]) {
74         valid = false;
75         break;
76     } else if (scc[u] < scc[u +
        y]) {
77         s += "+ ";
78     } else {
79         s += "- ";
80     }
81 }
82 cout << (valid ? s : "
    IMPOSSIBLE") << '\n';
83
84 return 0;
85 }

```

6.11 Topo-sort DFS

```

1 ll v;
2 vector<ll> topo;
3 vector<bool> vis;
4 vector<vector<ll>> adj(v + 1);
5 void dfs(ll node) {
6     vis[node] = true;
7     for (ll neighbor : adj[node]) {
8         if (vis[neighbor]) continue;
9         dfs(neighbor);
10    }
11    topo.pb(node);
12 }
13
14 reverse(topo.begin(), topo.end());

```

6.12 Dinic 無權二分圖最大 匹配

```

1 // 輸出無權二分圖最大匹配的邊數量和
    選到的邊
2
3 #include <bits/stdc++.h>
4 using namespace std;
5 using ll = long long;

```

```

6 #define pb push_back
7 constexpr ll INF = 1e18;
8
9 struct Node {
10     ll nd, pvIdx, cap;
11     Node(ll _nd, ll _pvIdx, ll _cap
        ) : nd(_nd), pvIdx(_pvIdx)
        , cap(_cap) {}
12 };
13
14 ll n1, n2, n, m, st, en;
15 vector<ll> level, iter;
16 vector<vector<Node>> adj;
17
18 void bfs() {
19     level.clear(); level.resize(n +
        1, -1);
20     level[st] = 0;
21
22     queue<ll> q; q.push(st);
23     while (not q.empty()) {
24         ll sz = q.size();
25         for (ll _ = 1; _ <= sz; ++_
            ) {
26             ll u = q.front(); q.pop
                ();
27             for (const Node& v :
                adj[u]) {
28                 if (v.cap > 0 and
                    level[v.nd] ==
                    -1) {
29                     level[v.nd] =
                        level[u] +
                        1;
30                     q.push(v.nd);
31                 }
32             }
33         }
34     }
35 }
36
37 ll dfs(ll u, ll flow) {
38     if (u == en) return flow;
39     for (ll& i = iter[u]; i < adj[u]
        .size(); ++i) {
40         Node& v = adj[u][i];
41         if (v.cap > 0 and level[v.
            nd] == level[u] + 1) {
42             ll fw = dfs(v.nd, min(
                flow, v.cap));
43             if (fw > 0) {
44                 v.cap -= fw;
45                 adj[v.nd][v.pvIdx].
                    cap += fw;
46                 return fw;
47             }
48         }
49     }
50     return 0; // for compile
        warning
51 }
52
53 ll maxFlow() {
54     ll res = 0;
55     while (true) {
56         bfs();

```

```

57     if (level[en] == -1) break;
58     ll cur;
59     iter.clear(); iter.resize(n
60         + 1);
61     while ((cur = dfs(st, INF))
62         > 0) {
63         res += cur;
64     }
65     return res;
66 }
67
68 int main() {
69     ios::sync_with_stdio(false);
70     cin.tie(nullptr);
71
72     cin >> n1 >> n2 >> m;
73     n = n1 + n2 + 2;
74     st = n1 + n2 + 1;
75     en = n1 + n2 + 2;
76     adj.clear(); adj.resize(n + 1);
77     for (ll i = 1; i <= m; ++i) {
78         ll u, v; cin >> u >> v; v
79             += n1;
80         adj[u].pb(Node(v, adj[v].
81             size(), 1));
82         adj[v].pb(Node(u, adj[u].
83             size() - 1, 0));
84     }
85     m += n1 + n2;
86     for (ll i = 1; i <= n1; ++i) {
87         adj[st].pb(Node(i, adj[i].
88             size(), 1));
89         adj[i].pb(Node(st, adj[st].
90             size() - 1, 0));
91     }
92     cout << maxFlow() << '\n';
93     for (ll u = 1; u <= n1; ++u) {
94         for (const Node& v : adj[u]
95             ) {
96             if (v.nd != st and v.
97                 cap == 0) {
98                 cout << u << ' ' <<
99                     v.nd - n1 <<
100                     '\n';
101         }
102     }
103     return 0;

```

6.13 Dinic Min Cut

```

1 // 最小割是選一些邊可以阻斷所有水流
2 且這些邊的容量和最小
3 // 輸出最小割的容量和選到的邊
4 // (最大流容量和 == 最小割容量和)
5 #include <bits/stdc++.h>
6 using namespace std;
7 using ll = long long;
8 #define pb push_back
9 constexpr ll INF = 1e18;
10
11 struct Node {
12     bool isPseudo;
13     ll nd, pvIdx, cap;
14     Node(ll _nd, ll _pvIdx, ll _cap
15         , bool _isPseudo) : nd(_nd
16             ), pvIdx(_pvIdx), cap(_cap
17             ), isPseudo(_isPseudo) {}
18 };
19 ll n, m, st, en;
20 vector<ll> level, iter;
21 vector<vector<Node>> adj;
22
23 void bfs() {
24     level.clear(); level.resize(n +
25         1, -1);
26     level[st] = 0;
27
28     queue<ll> q; q.push(st);
29     while (not q.empty()) {
30         ll sz = q.size();
31         for (ll _ = 1; _ <= sz; ++_
32             ) {
33             ll u = q.front(); q.pop
34                 ();
35             for (const Node& v :
36                 adj[u]) {
37                 if (v.cap > 0 and
38                     level[v.nd] ==
39                     -1) {
40                     level[v.nd] =
41                         level[u] +
42                         1;
43                     q.push(v.nd);
44                 }
45             }
46         }
47     }
48
49     ll dfs(ll u, ll flow) {
50         if (u == en) return flow;
51         for (ll& i = iter[u]; i < adj[u]
52             .size(); ++i) {
53             Node& v = adj[u][i];
54             if (v.cap > 0 and level[v.
55                 nd] == level[u] + 1) {
56                 ll fw = dfs(v.nd, min(
57                     flow, v.cap));
58                 if (fw > 0) {
59                     v.cap -= fw;
60                     adj[v.nd][v.pvIdx].
61                         cap += fw;
62                     return fw;

```

```

50     }
51     }
52     }
53     return 0; // for compile
54     warning
55 }
56 ll maxFlow() {
57     ll res = 0;
58     while (true) {
59         bfs();
60         if (level[en] == -1) break;
61         ll cur;
62         iter.clear(); iter.resize(n
63             + 1);
64         while ((cur = dfs(st, INF))
65             > 0) {
66             res += cur;
67         }
68     }
69     return res;
70 }
71 vector<bool> vis;
72 void markSetS(ll u) {
73     vis[u] = true;
74     for (const Node& v : adj[u]) {
75         if (v.isPseudo) continue;
76         if (not vis[v.nd] and v.cap
77             > 0) {
78             markSetS(v.nd);
79         }
80     }
81 }
82 void outputCut() {
83     for (ll u = 1; u <= n; ++u) {
84         if (not vis[u]) continue;
85         // set T
86         for (const Node& v : adj[u]
87             ) {
88             if (v.isPseudo)
89                 continue;
90             if (not vis[v.nd]) {
91                 cout << u << ' ' <<
92                     v.nd << '\n';
93             }
94         }
95     }
96 }
97 int main() {
98     ios::sync_with_stdio(false);
99     cin.tie(nullptr);
100
101     cin >> n >> m;
102     adj.clear(); adj.resize(n + 1);
103     for (ll i = 1; i <= m; ++i) {
104         ll u, v; cin >> u >> v;
105         adj[u].pb(Node(v, adj[v].
106             size(), 1, false));
107         adj[v].pb(Node(u, adj[u].
108             size() - 1, 0, true));

```

```

106     adj[v].pb(Node(u, adj[u].
107         size(), 1, false));
108     adj[u].pb(Node(v, adj[v].
109         size() - 1, 0, true));
110 }
111 st = 1;
112 en = n;
113 cout << maxFlow() << '\n'; //
114     maxFlow == minCut
115
116 vis.resize(n + 1);
117 markSetS(st);
118 outputCut();
119 return 0;

```

6.14 Floyd Warshall

```

1 int main() {
2     ios::sync_with_stdio(false);
3     cin.tie(nullptr);
4
5     cin >> n >> m;
6     mtx.clear(); mtx.resize(n + 1,
7         vector<ll>(n + 1, INF));
8     for (ll i = 1; i <= n; ++i) mtx[i]
9         [[i] = 0;
10     rep (m) {
11         ll x, y, w; cin >> x >> y >> w;
12         mtx[x][y] = min(mtx[x][y], w);
13         mtx[y][x] = min(mtx[y][x], w);
14     } // "min" is used to prevent
15         the multiple edges
16
17     for (ll k = 1; k <= n; ++k) {
18         for (ll i = 1; i <= n; ++i) {
19             for (ll j = 1; j <= n; ++j) {
20                 mtx[i][j] = min(mtx[i][j],
21                     mtx[i][k] + mtx[k][j]);

```

6.15 MST Kruskal

```

1 vector<ll> djs, sz;
2 ll find(ll u) {...}
3 void join(ll u, ll v) {...}
4 void init(ll n) {...}
5
6 class Edge{
7 public:
8     ll u, v, w;
9 };
10
11 int main() {
12     ios::sync_with_stdio(false);

```

```

13     cin.tie(nullptr);
14
15     // nodes are 1-indexed
16     // edges are 0-indexed
17
18     ll n, m; cin >> n >> m;
19     vector<Edge> edges(m);
20     for (auto& [u, v, w] : edges) {
21         cin >> u >> v >> w;
22     }
23     sort(all(edges), [](const Edge&
24         e1, const Edge e2) ->
25         bool {
26             return e1.w < e2.w;
27         });
28     init(n);
29
30     ll mstCost = 0, mstEdgesCnt =
31         0;
32     for (const auto& [u, v, w] :
33         edges) {
34         if (find(u) == find(v))
35             continue;
36         join(u, v);
37         mstCost += w;
38         mstEdgesCnt += 1;
39         if (mstEdgesCnt == n - 1)
40             break;
41     }
42
43     // if (mstEdgesCnt < n - 1) //
44         the graph is disconnected
45         (MST D.N.E.)
46
47     return 0;

```

6.16 Tarjan 縮點

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using pll = pair<ll, ll>;
5 #define pb push_back
6
7 ll n, m, t = 0, sccCnt = 0, sccIdx
8     = 0;
9 stack<ll> stk;
10 vector<bool> inStack;
11 vector<ll> in, low, scc, val;
12 vector<vector<ll>> adj;
13 void dfs(ll u) {
14     low[u] = in[u] = ++t;
15     stk.push(u);
16     inStack[u] = true;
17     for (ll v : adj[u]) {
18         if (in[v] == -1) {
19             dfs(v);
20             low[u] = min(low[u],
21                 low[v]);
22         } else if (inStack[v]) {
23             low[u] = min(low[u],
24                 low[v]);

```



```

22     }
23 }
24 if (in[u] == low[u]) {
25     sccCnt += 1;
26     sccIdx += 1;
27     ll cur;
28     do {
29         cur = stk.top(); stk.
30             pop();
31         inStack[cur] = false;
32         scc[cur] = sccIdx;
33     } while (cur != u);
34 }
35 vector<bool> vis2;
36 vector<ll> val2, dp2;
37 vector<vector<ll>> adj2;
38 void dfs2(ll u) {
39     vis2[u] = true;
40     for (ll v : adj2[u]) {
41         if (not vis2[v]) dfs2(v);
42         dp2[u] = max(dp2[u], val2[u]
43             + dp2[v]);
44     }
45 }
46 int main() {
47     ios::sync_with_stdio(false);
48     cin.tie(nullptr);
49
50     cin >> n >> m;
51     inStack.resize(n + 1);
52     in.resize(n + 1, -1);
53     low.resize(n + 1);
54     scc.resize(n + 1);
55     val.resize(n + 1);
56     adj.resize(n + 1);
57     for (ll u = 1; u <= n; ++u) cin
58         >> val[u];
59     for (ll i = 1; i <= m; ++i) {
60         ll u, v; cin >> u >> v;
61         adj[u].pb(v);
62     }
63
64     for (ll u = 1; u <= n; ++u) {
65         if (in[u] != -1) continue;
66         dfs(u);
67     }
68
69     vis2.resize(sccCnt + 1);
70     val2.resize(sccCnt + 1);
71     dp2.resize(sccCnt + 1);
72     adj2.resize(sccCnt + 1);
73     for (ll u = 1; u <= n; ++u) {
74         val2[scc[u]] += val[u];
75         for (ll v : adj[u]) {
76             if (scc[v] == scc[u])
77                 continue;
78             adj2[scc[u]].pb(scc[v]);
79         }
80     }
81     for (ll u = 1; u <= sccCnt; ++u)
82         dp2[u] = val2[u];

```

```

82     }
83
84     for (ll u = 1; u <= sccCnt; ++u)
85         {
86             if (vis2[u]) continue;
87             dfs2(u);
88         }
89     cout << *max_element(dp2.begin
90         () + 1, dp2.end()) << '\n';
91     return 0;
92 }

```

6.17 Kosaraju

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 #define pb push_back
5 #define rep(n) for (ll _ = 1; _ <=
6     n; ++_)
7 ll V, E;
8
9 stack<ll> stk;
10 vector<bool> vis;
11 vector<vector<ll>> adj;
12 void dfs1(ll u) {
13     vis[u] = true;
14     for (ll v : adj[u]) {
15         if (vis[v]) continue;
16         dfs1(v);
17     }
18     stk.push(u);
19 }
20 ll sccCnt;
21 vector<ll> scc;
22 vector<vector<ll>> radj;
23 void dfs2(ll u, ll sccIdx) {
24     scc[u] = sccIdx;
25     for (ll v : radj[u]) {
26         if (scc[v] != -1) continue;
27         dfs2(v, sccIdx);
28     }
29 }
30 }
31
32 int main() {
33     ios::sync_with_stdio(false);
34     cin.tie(nullptr);
35
36     cin >> V >> E;
37     vis.resize(V + 1, false);
38     adj.resize(V + 1);
39     radj.resize(V + 1);
40     rep(E) {
41         ll x, y; cin >> x >> y;
42         adj[x].pb(y);
43         radj[y].pb(x);
44     }
45 }

```

```

46 for (ll u = 1; u <= V; ++u) {
47     if (vis[u]) continue;
48     dfs1(u);
49 }
50 sccCnt = 0;
51 scc.resize(V + 1, -1);
52 while (not stk.empty()) {
53     ll u = stk.top(); stk.pop()
54         ;
55     if (scc[u] != -1) continue;
56     dfs2(u, ++sccCnt);
57 }
58
59 cout << sccCnt << '\n';
60 for (ll i = 1; i <= V; ++i) {
61     cout << scc[i] << ' ';
62 } cout << '\n';
63
64 return 0;
65 }

```

7 Math

7.1 Fast Exponentiation

```

1 ll fastPow(ll b, ll e) {
2     b = b%MOD;
3     ll res = 1;
4     while (e > 0) {
5         if (e%2 == 1) res = (res*b)%MOD
6             ;
7         b = (b*b)%MOD;
8         e /= 2;
9     }
10    return res;

```

7.2 Combinations

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int M = 1e9 + 7;
5 const int N = 200005;
6 vector<int> fact(N), invfact(N);
7
8 int qpow(int a, int b){
9     int res=1;
10    while(b > 0){
11        if(b & 1) res = res * a % M
12            ;
13        a = a * a % M;
14        b >>= 1;
15    }
16    return res;
17 }

```

```

18 void dofact(){
19     fact[0] = 1;
20     for (int i = 1; i < N; i++) {
21         fact[i] = fact[i - 1] * i %
22             M;
23     }
24     invfact[N - 1] = qpow(fact[N -
25         1], M - 2);
26     for(int i = N - 1; i > 0; i--)
27         invfact[i - 1] = invfact[i]
28             * i % M;
29 }
30
31 int C(int n,int m){
32     return fact[n] * invfact[m] % M
33         * invfact[n - m] % M;
34 }
35
36 void solve() {
37
38     signed main() {
39         // FIO;
40         int t_t = 1;
41         // cin >> t_t;
42         while (t_t--)
43             solve();
44     }
45
46     //
47     // =====
48
49     // =====
50
51     ll MAXN;
52     vector<ll> fact; // factorial
53     vector<ll> invs; // modular
54         inverse
55     void init() {
56         fact.resize(MAXN + 1, 1);
57         invs.resize(MAXN + 1, 1);
58         for (ll i = 2; i <= MAXN; ++i)
59             {
60                 fact[i] = (fact[i - 1]*i)%
61                     MOD;
62                 invs[i] = fastPow(fact[i],
63                     MOD - 2);
64             }
65     }
66
67     inline ll C(ll n, ll k) {
68         return fact[n]*invs[k]%MOD*invs[n
69             - k]%MOD
70     }

```

7.3 Matrix

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct Matrix {
5     int n, m; // n
6     // 行, m 列
7     vector<vector<long long>> a;
8     static const long long MOD = 1
9         e9+7; // 如果題目需要取
10         模 · 可以改這裡
11
12     Matrix(int n, int m, bool ident
13         = false): n(n), m(m) {
14         a.assign(n, vector<long
15             long>(m, 0));
16         if(ident) { // 單位矩陣
17             for(int i=0; i<min(n,m)
18                 ; i++) a[i][i] =
19                 1;
20         }
21     }
22
23     // 輸出矩陣
24     void print() const {
25         for(int i=0; i<n; i++) {
26             for(int j=0; j<m; j++)
27                 cout << a[i][j] <<
28                     " ";
29             cout << "\n";
30         }
31     }
32
33     // 矩陣乘法
34     Matrix operator*(const Matrix&
35         o) const {
36         assert(m == o.n);
37         Matrix res(n, o.m);
38         for(int i=0; i<n; i++) {
39             for(int k=0; k<m; k++)
40                 if(a[i][k]) {
41                     for(int j=0; j<o.m;
42                         j++) {
43                         res.a[i][j] = (
44                             res.a[i][j]
45                             + a[i][k]
46                             * o.a[k]
47                             ][j]) %
48                             MOD;
49                     }
50                 }
51             }
52         return res;
53     }
54
55     // 矩陣加法
56     Matrix operator+(const Matrix&
57         o) const {
58         assert(n == o.n && m == o.m
59             );
60         Matrix res(n, m);
61         for(int i=0; i<n; i++) {

```

```

43     for(int j=0; j<m; j++)
44     {
45         res.a[i][j] = (a[i
46             ][j] + o.a[i][
47                 j]) % MOD;
48     }
49     return res;
50 }
51 // 矩陣快速冪 (n×n 方陣才能做)
52 Matrix pow(long long exp) const
53 {
54     assert(n == m);
55     Matrix res(n, n, true),
56     base = *this;
57     while(exp > 0) {
58         if(exp & 1) res = res *
59         base;
60         base = base * base;
61         exp >>= 1;
62     }
63     return res;
64 }

```

7.4 Modular Inverse

```

1 // extend Euclidean
2 pll extgcd(ll a, ll b) {
3     if (b == 0) return pll(1, 0);
4     pll p = extgcd(m, a%b);
5     ll x = p.ff, y = p.ss;
6     return pll(y, x - y*(a/b));
7 }
8 extgcd(a, MOD).ff // the modular
9 inverse
10 (extgcd(a, MOD).ff%MOD + MOD)%MOD
11 // if you want to ensure that
12 it is non-negative
13 // fast exponentiation (make sure
14 that MOD is a prime number)
15 fastPow(a, MOD - 2) // the modular
16 inverse

```

7.5 Big Integer Addition and Multiplication

```

1 vector<int> strToVec(string str,
2 int sz) {
3     vector<int> r(sz, 0);
4     int strlength = str.length();
5     for (int i = strlength - 1, idx
6         = 0; i >= 0; --i, ++idx)
7     {
8         r[idx] = str[i] - '0';
9     }
10    return r;

```

```

8 }
9 // for example:
10 // strToVec("677", 4) -> 7 7 6 0
11 // strToVec("8829", 4) -> 9 2 8 8
12 // addition
13 string add(string x, string y) {
14     ll n = max(x.length(), y.length
15         ());
16     vector<ll> xdigit = strToVec(x,
17         n + 1);
18     vector<ll> ydigit = strToVec(y,
19         n + 1);
20     vector<ll> result(n + 1, 0);
21     ll carry = 0;
22     for (ll i = 0; i < n + 1; ++i)
23     {
24         result[i] = xdigit[i] +
25         ydigit[i] + carry;
26         if (result[i] >= 10) {
27             result[i] %= 10;
28             carry = 1;
29         }
30         else carry = 0;
31     }
32     string r = "";
33     for (ll i = start; i >= 0; --i)
34     {
35         r += result[i] + '0';
36     }
37     return r;
38 }
39 // multiplication
40 string product(string x, string y)
41 {
42     ll xlength = x.length();
43     ll ylength = y.length();
44     ll n = max(xlength, ylength);
45     vector<ll> xdigit = strToVec(x,
46         xlength);
47     vector<ll> ydigit = strToVec(y,
48         ylength);
49     vector<ll> result(2*n, 0);
50     for (ll i = 0; i < xlength; ++i)
51     {
52         for (ll j = 0; j < ylength;
53             ++j) {
54             result[i + j] += xdigit
55             [i]*ydigit[j];
56             if (result[i + j] >=
57                 10) {
58                 result[i + j + 1]
59                 += result[i +
60                     j]/10;
61                 result[i + j] %=
62                 10;
63             }
64         }
65     }
66 }

```

```

58 }
59 ll start;
60 for (ll i = 2*n - 1; i >= 0; --
61     i) {
62     if (result[i] != 0) {
63         start = i;
64         break;
65     }
66     string r = "";
67     for (ll i = start; i >= 0; --i)
68     {
69         r += result[i] + '0';
70     }
71     return r;
72 }

```

7.6 Prime Sieve

```

1 // Sieve of Eratosthenes
2 ll MAXN;
3 vector<ll> spf(MAXN + 1, -1);
4 spf[0] = -2; spf[1] = -2;
5 for (ll i = 2; i*i <= MAXN; ++i) {
6     if (spf[i] == -1) {
7         for (ll j = i*i; j <= MAXN;
8             j += i) {
9             if (spf[j] == -1) {
10                 spf[j] = i;
11             }
12         }
13     }
14 }

```

8 Others

8.1 mt19937 Usage

```

1 #include <chrono>
2 #include <random>
3 using namespace std;
4 using ll = long long;
5 // mt19937 is a random number
6 generator
7 // chrono::steady_clock::now().
8 time_since_epoch().count() is
9 used as a seed
10 mt19937 rng(chrono::steady_clock::
11 now().time_since_epoch().count
12 ());
13 // dist describes a range of random
14 numbers
15 uniform_int_distribution<ll> dist(
16     lb, ub);
17 }

```

```

13 dist(rng) // use rng to generate a
14 random integer in [lb, ub]

```

8.2 GCC Builtin Functions

```

1 // count the number of 1 bit in x
2 __builtin_popcount(unsigned int x)
3 __builtin_popcountll(unsigned long
4 long x)
5 // count leading zero of x
6 __builtin_clz(unsigned int x)
7 __builtin_clzll(unsigned long long
8 x)
9 // count trailing zero of x
10 __builtin_ctz(unsigned int x)
11 __builtin_ctzll(unsigned long long
12 x)

```

8.3 distinct k number

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define int long long
4 #define all(c) (c).begin(), (c).end
5 ()
6 void solve() {
7     int n, k, L, R;
8     cin >> n >> k >> L >> R;
9     vector<int> uni(n);
10    vector<int> vec(n);
11    for (int i = 0; i < n; i++) {
12        cin >> uni[i];
13        vec[i] = uni[i];
14    }
15    sort(all(uni));
16    uni.resize(unique(all(uni)) -
17        uni.begin());
18    vector<int> f_low(uni.size(),
19        0);
20    vector<int> f_up(uni.size(), 0)
21    ;
22    for (int i = 0; i < n; i++) {
23        vec[i] = lower_bound(all(
24            uni), vec[i]) - uni.
25            begin();
26    }
27    int l = -1, r = 0;
28    int cl = 0, cr = 0;
29    int res = 0;
30    for (int i = 0; i < n; i++) {
31        while (r < n && cr <= k) {
32            if (f_low[vec[r]] == 0)
33                if (cr == k) break;
34            cr++;
35        }
36        f_up[vec[r++]]++;
37    }

```

```

32 }
33 while (l + 1 < n && cl < k)
34 {
35     if (f_low[vec[l + 1]]
36         == 0) {
37         cl++;
38     }
39     f_low[vec[l + 1]]++;
40     l++;
41 }
42 if (cl == k) {
43     res += max(min(r, i + R
44         ) - max(l, i + L -
45             1), 0LL);
46 }
47 f_low[vec[i]]--;
48 f_up[vec[i]]--;
49 if (f_low[vec[i]] == 0) cl
50 --;
51 if (f_up[vec[i]] == 0) cr
52 --;
53 }
54 cout << res << '\n';
55 }

```

8.4 Custom Hash

```

1 #include <chrono>
2 // =====
3 // =====
4 class custom_hash {
5 private:
6     static uint64_t splitmix64(
7         uint64_t x) {
8         x += 0x9e3779b97f4a7c15;
9         x = (x ^ (x >> 30)) * 0
10             xbf58476d1ce4e5b9;
11         x = (x ^ (x >> 27)) * 0
12             x94d049bb133111eb;
13         return x ^ (x >> 31);
14     }
15 public:
16     size_t operator()(uint64_t x)
17     const {
18         static const uint64_t
19             FIXED_RANDOM = std:::
20             chrono::steady_clock::
21             now().time_since_epoch
22             ().count();
23         return splitmix64(x +
24             FIXED_RANDOM);
25     }
26 }
27 // for example:
28 unordered_map<ll, ll,
29 custom_hash> mp;
30 // =====
31 // =====

```

```

26 class custom_hash {
27 private:
28     static const uint64_t
        FIXED_RANDOM;
29     static uint64_t splitmix64(
        uint64_t x) {
30         x += 0x9e3779b97f4a7c15;
31         x = (x ^ (x >> 30)) * 0
            xbf58476d1ce4e5b9;
32         x = (x ^ (x >> 27)) * 0
            x94d049bb133111eb;
33         return x ^ (x >> 31);
34     }
35 public:
36     size_t operator()(uint64_t x)
        const {
37         return splitmix64(x +
            FIXED_RANDOM);
38     }
39     size_t operator()(pair<uint64_t
        , uint64_t> p) const {
40         uint64_t h1 = splitmix64(p.
            first + FIXED_RANDOM);
41         uint64_t h2 = splitmix64(p.
            second + FIXED_RANDOM
            + 1);
42         return h1 ^ (h2*47);
43     }
44 };
45 const uint64_t custom_hash::
        FIXED_RANDOM = std::chrono::
        steady_clock::now().
        time_since_epoch().count();
46 // for example:
47 // unordered_map<pLL, LL,
        custom_hash> mp;

```

9 String

9.1 Z-value

```

1 // CSES: String Matching
2 // Given a string and a pattern,
  your task is to count
3 // the number of positions where
  the pattern occurs in the
  string.
4
5 #include <bits/stdc++.h>
6 using namespace std;
7 using ll = long long;
8
9 vector<ll> z_value(const string& s)
10 {
11     ll n = s.size();
12     vector<ll> z(n);
13     ll l = 0, r = 0;
14     for (ll i = 1; i < n; ++i) {
15         if (i <= r) z[i] = min(z[i
            - l], r - i + 1);
16     }
17     return z;
18 }

```

```

15 while (i + z[i] < n && s[z[
    i]] == s[i + z[i]]) z[
    i] += 1;
16 if (i + z[i] - 1 > r) {
17     l = i;
18     r = i + z[i] - 1;
19 }
20 }
21 return z;
22 }
23
24 int main() {
25     ios::sync_with_stdio(false);
26     cin.tie(nullptr);
27
28     string s1, s2; cin >> s1 >> s2;
29     ll n = s1.size(), m = s2.size()
        ;
30
31     ll cnt = 0;
32     string s = s2 + "$" + s1;
33     vector<ll> z = z_value(s);
34     for (ll i = m; i < s.size(); ++
        i) {
35         if (z[i] == m) cnt += 1;
36     }
37     cout << cnt << '\n';
38
39     return 0;
40 }
41
42 // =====
43 // =====
44
45 // CSES: Finding Borders
46 // A border of a string is a prefix
  that is also a suffix of
47 // the string but not the whole
  string. For example,
48 // the borders of abcababca are ab
  and abcab.
49 // Your task is to find all border
  lengths of a given string.
50
51 #include <bits/stdc++.h>
52 using namespace std;
53 using ll = long long;
54 #define pb push_back
55
56 vector<ll> z_value(const string& s)
57 {
58     ll n = s.size();
59     vector<ll> z(n);
60     ll l = 0, r = 0;
61     for (ll i = 1; i < n; ++i) {
62         if (i <= r) z[i] = min(z[i
            - l], r - i + 1);
63         while (i + z[i] < n && s[z[
            i]] == s[i + z[i]]) z[
            i] += 1;
64         if (i + z[i] - 1 > r) {
65             l = i;
66             r = i + z[i] - 1;
67         }
68     }
69     return z;
70 }

```

```

71 }
72
73 int main() {
74     ios::sync_with_stdio(false);
75     cin.tie(nullptr);
76
77     string s; cin >> s;
78     ll n = s.size();
79
80     vector<ll> z = z_value(s);
81     for (ll i = 1; i < n; ++i) {
82         if (i + z[i] == n) res.pb(z
            [i]);
83     }
84     sort(res.begin(), res.end());
85     for (ll x : res) {
86         cout << x << ' ';
87     } cout << '\n';
88
89     return 0;
90 }
91
92 // =====
93 // =====
94
95 // CSES: Finding Periods
96 // A period of a string is a prefix
  that can be used to generate
97 // the whole string by repeating
  the prefix. The last
  repetition
98 // may be partial. For example, the
  periods of abcabca are abc,
  abcab and abcabca.
99 // Your task is to find all period
  lengths of a string.
100
101 #include <bits/stdc++.h>
102 using namespace std;
103 using ll = long long;
104 #define pb push_back
105
106 vector<ll> z_value(const string& s)
107 {
108     ll n = s.size();
109     vector<ll> z(n);
110     ll l = 0, r = 0;
111     for (ll i = 1; i < n; ++i) {
112         if (i <= r) z[i] = min(z[i
            - l], r - i + 1);
113         while (i + z[i] < n && s[z[
            i]] == s[i + z[i]]) z[
            i] += 1;
114         if (i + z[i] - 1 > r) {
115             l = i;
116             r = i + z[i] - 1;
117         }
118     }
119     return z;
120 }
121
122 int main() {
123     ios::sync_with_stdio(false);
124     cin.tie(nullptr);
125
126     string s; cin >> s;
127 }

```

```

124 ll n = s.size();
125
126 vector<ll> z = z_value(s);
127 for (ll i = 1; i < n; ++i) {
128     if (i + z[i] == n) res.pb(i
        );
129 }
130 sort(res.begin(), res.end());
131 for (ll x : res) {
132     cout << x << ' ';
133 } cout << '\n';
134
135 return 0;
136 }

```

9.2 KMP

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using pll = pair<ll, ll>;
5 template<class T> using vec =
    vector<T>;
6 #define endl '\n'
7 #define ff first
8 #define ss second
9 #define pb push_back
10 #define rep(n) for (ll _ = 1; _ <=
    n; ++_)
11
12 // -----
13
14 vector<ll> LPS(const string& s) {
15     // 0-indexed
16     ll n = s.size();
17     if (n == 0) return {};
18     vector<ll> lps(n); lps[0] = 0;
19     for (ll i = 1, j = 0; i < n; ++
        i) {
20         while (j > 0 && s[i] != s[j]
            ) j = lps[j - 1];
21         if (s[i] == s[j]) j += 1;
22         lps[i] = j;
23     }
24     return lps;
25 }
26
27 ll KMP(const string& s, const
    string& t) { // 0-indexed
28     vector<ll> lps = LPS(t);
29     ll n = s.size(), m = t.size();
30     if (n < m) return 0;
31     ll i = 0, j = 0, cnt = 0;
32     while (i < n) {
33         if (s[i] == t[j]) {
34             i += 1;
35             j += 1;
36         } else if (j > 0) {
37             j = lps[j - 1];
38         } else if (j == 0) {
39             i += 1;
40         }
41     }
42     return cnt;
43 }

```

```

39 }
40 if (j == m) {
41     // s[i - m to i - 1] ==
        t
42     cnt += 1;
43     j = lps[j - 1];
44 }
45 }
46 return cnt; // the number of
    matching substrings
47 }
48
49 int main() {
50     ios::sync_with_stdio(false);
51     cin.tie(nullptr);
52
53     string s, t;
54     cin >> s >> t;
55
56     cout << KMP(s, t) << endl;
57
58     return 0;
59 }

```

9.3 Trie

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class Trie {
5     struct Node {
6         bool endofWord;
7         vector<int> children;
8         Node() : endofWord(false),
            children(26, -1) {}
9     };
10
11     vector<Node> trie;
12
13 public:
14     Trie() {
15         trie.emplace_back();
16     }
17
18     void insert(const string& word)
19     {
20         int cur = 0;
21         for (char c : word) {
22             int idx = c - 'a';
23             if (trie[cur].children[
                idx] == -1) {
24                 trie[cur].children[
                    idx] = trie.
                    size();
25                 trie.emplace_back()
                ;
26                 cur = trie[cur].
                    children[idx];
27             }
28             trie[cur].endofWord = true;
29         }
30 }

```

```

31 bool search(const string& word)
32 {
33     int cur = 0;
34     for (char c : word) {
35         int idx = c - 'a';
36         if (trie[cur].children[
37             idx] == -1) return
38             false;
39         cur = trie[cur].
40             children[idx];
41     }
42     return trie[cur].endofWord;
43 }
44
45 bool startsWith(const string&
46     prefix) {
47     int cur = 0;
48     for (char c : prefix) {
49         int idx = c - 'a';
50         if (trie[cur].children[
51             idx] == -1) return
52             false;
53         cur = trie[cur].
54             children[idx];
55     }
56     return true;
57 }
58
59 void deleteWord(const string&
60     word) {
61     int cur = 0;
62     for (char c : word) {
63         int idx = c - 'a';
64         if (trie[cur].children[
65             idx] == -1) return
66             ;
67         cur = trie[cur].
68             children[idx];
69     }
70     trie[cur].endofWord = false
71     ;
72 }
73
74 void print(int node, string
75     prefix) const {
76     if (trie[node].endofWord) {
77         cout << prefix << "\n";
78     }
79     for (int i = 0; i < 26; i
80         ++){
81         int nxt = trie[node].
82             children[i];
83         if (nxt != -1) {
84             print(nxt, prefix +
85                 char('a' + i)
86             );
87         }
88     }
89 }
90
91 void print() const { print(0, ""
92     ); }
93
94 int main() {
95     Trie trie;

```

```

96     trie.insert("geek");
97     trie.insert("geeks");
98     trie.insert("code");
99     trie.insert("coder");
100    trie.insert("coding");
101
102    cout << "Trie contents:\n";
103    trie.print();
104
105    cout << "\nSearch results:\n";
106    cout << "geek: " << trie.search
107        ("geek") << "\n";
108    cout << "geeks: " << trie.
109        search("geeks") << "\n";
110    cout << "code: " << trie.search
111        ("code") << "\n";
112    cout << "coder: " << trie.
113        search("coder") << "\n";
114    cout << "coding: " << trie.
115        search("coding") << "\n";
116    cout << "codex: " << trie.
117        search("codex") << "\n";
118
119    cout << "\nPrefix results:\n";
120    cout << "ge: " << trie.
121        startsWith("ge") << "\n";
122    cout << "cod: " << trie.
123        startsWith("cod") << "\n";
124    cout << "coz: " << trie.
125        startsWith("coz") << "\n";
126
127    trie.deleteWord("coding");
128    trie.deleteWord("geek");
129
130    cout << "\nTrie contents after
131        deletions:\n";
132    trie.print();
133
134    cout << "\nSearch results after
135        deletions:\n";
136    cout << "coding: " << trie.
137        search("coding") << "\n";
138    cout << "geek: " << trie.search
139        ("geek") << "\n";
140
141    return 0;

```

9.4 Manacher

```

1 // CSES: Longest Palindrome
2 // Given a string, your task is to
3 // determine the longest
4 // palindromic substring of the
5 // string. For example,
6 // the longest palindrome in
7 // ayabatu is bab.

```

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;

```

```

10 int main() {
11     ios::sync_with_stdio(false);
12     cin.tie(nullptr);
13
14     string t, s = "^#"; cin >> t;
15     ll n = t.size(), m = 2*n + 3;
16     for (ll i = 0; i < n; ++i) {
17         s += t[i];
18         s += (i == n - 1 ? "$" :
19             "#");
20     }
21
22     ll c = 1;
23     vector<ll> p(m); p[1] = 0;
24     for (ll i = 2; i <= m - 3; ++i)
25     {
26         if (i < c + p[c]) p[i] =
27             min(p[c] - (i - c), (c
28                 + p[c]) - i);
29         while (i + p[i] + 1 < m &&
30             i - p[i] - 1 >= 0 &&
31             s[i - p[i] - 1] == s
32                 [i + p[i] + 1])
33             p[i] += 1;
34         if (i + p[i] > c + p[c]) c
35             = i;
36     }
37
38     ll j = 2;
39     for (ll i = 3; i <= m - 3; ++i)
40     {
41         if (p[i] > p[j]) j = i;
42     }
43
44     for (ll i = j - p[j] + 1; i <=
45         j + p[j] - 1; i += 2) {
46         cout << s[i];
47     } cout << '\n';
48
49     return 0;
50 }
51
52 // =====
53 // CSES: ALL Palindromes
54 // Given a string, calculate for
55 // each position the length
56 // of the longest palindrome that
57 // ends at that position.
58
59 #include <bits/stdc++.h>
60 using namespace std;
61 using ll = long long;
62
63 ll n, m;
64 string ori_s, s;
65 vector<ll> p, rt, dp;
66
67 int main() {
68     ios::sync_with_stdio(false);
69     cin.tie(nullptr);
70
71     cin >> ori_s;
72     n = ori_s.size();
73     m = 2*n + 3;

```

```

65 s = "^#";
66 for (ll i = 0; i < n; ++i) {
67     s += ori_s[i];
68     s += (i == n - 1 ? "$" :
69         "#");
70 }
71
72 ll c = 0;
73 p.resize(m);
74 for (ll i = 2; i <= m - 3; ++i)
75 {
76     if (i < c + p[c]) p[i] =
77         min(p[c] - (i - c), (c
78             + p[c]) - i);
79     while (i - p[i] - 1 >= 0 &&
80         i + p[i] + 1 < m &&
81         s[i - p[i] - 1] == s
82             [i + p[i] + 1])
83         p[i] += 1;
84     if (i + p[i] > c + p[c]) c
85         = i;
86 }
87
88 rt.resize(n, 1);
89 for (ll i = 2; i <= m - 3; ++i)
90 {
91     ll y = ((i + p[i] - 1) - 2)
92         / 2;
93     if (s[i] == '#') {
94         ll x = ((i + 1) - 2) / 2;
95         rt[y] = max(rt[y], (y -
96             x + 1) * 2);
97     } else {
98         ll x = (i - 2) / 2;
99         rt[y] = max(rt[y], (y -
100             x) * 2 + 1);
101     }
102 }
103
104 dp.resize(n); dp[n - 1] = rt[n
105     - 1];
106 for (ll i = n - 2; i >= 0; --i)
107 {
108     dp[i] = max(rt[i], dp[i +
109         1] - 2);
110 }
111
112 for (ll x : dp) {
113     cout << x << ' ';
114 } cout << '\n';
115
116 return 0;

```

9.5 String Hashing

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4 using pll = pair<ll, ll>;
5 #define pb push_back
6
7 class strHash {

```

```

8 private:
9     const ll m1 = 1e9 + 7, m2 = 1e9
10         + 9, p = 239017;
11     ll n;
12     string s;
13     vector<ll> h1, h2, p1, p2;
14 public:
15     strHash(const string& _s) {
16         n = _s.size();
17         s = _s;
18
19         h1.resize(n); h1[0] = s[0];
20         h2.resize(n); h2[0] = s[0];
21         for (ll i = 1; i < n; ++i)
22         {
23             h1[i] = (h1[i - 1]*p%
24                 m1
25                 + s[i])%m1;
26             h2[i] = (h2[i - 1]*p%
27                 m2
28                 + s[i])%m2;
29         }
30
31         p1.resize(n); p1[0] = 1;
32         p2.resize(n); p2[0] = 1;
33         for (ll i = 1; i < n; ++i)
34         {
35             p1[i] = p1[i - 1]*p%
36                 m1;
37             p2[i] = p2[i - 1]*p%
38                 m2;
39         }
40     }
41
42     pll hash(ll l, ll r) const {
43         // [l, r]
44         if (l == 0) return pll(h1[r],
45             h2[r]);
46         return pll(
47             ((h1[r] - h1[l - 1]*p1[
48                 r - l + 1]%m1)%m1
49                 + m1)%m1,
50             ((h2[r] - h2[l - 1]*p2[
51                 r - l + 1]%m2)%m2
52                 + m2)%m2
53         );
54     }
55 }
56
57 int main() {
58     ios::sync_with_stdio(false);
59     cin.tie(nullptr);
60
61     string s1, s2; cin >> s1 >> s2;
62     ll n = s1.size(), m = s2.size()
63     ;
64
65     if (n < m) {
66         cout << 0 << '\n';
67         return 0;
68     }
69
70     ll cnt = 0;
71     strHash sh1(s1), sh2(s2);
72     pll tarHash = sh2.hash(0, m -
73         1);
74     for (ll i = 0; i < n - m + 1;
75         ++i) {
76         if (sh1.hash(i, i + m - 1)
77             == tarHash) cnt += 1;
78     }

```

9.6 Suffix Array n log squared n version

```

59 |     cout << cnt << '\n';
60 |
61 |     return 0;
62 | }

1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | using ll = long long;
4 |
5 | vector<ll> suffix_array(const
   | string& s) {
6 |     ll n = s.size();
7 |     vector<ll> sa(n), rk(n), tmp(n)
   | ;
8 |
9 |     for (ll i = 0; i < n; ++i) {
10 |         sa[i] = i;
11 |         rk[i] = s[i] - 'a';
12 |     }
13 |
14 |     for (ll k = 1; k < n; k *= 2) {
15 |         auto cmp = [&](ll ri, ll li
   | ) {
16 |             if (rk[ri] != rk[li])
   |                 return rk[ri] < rk
   | [li];
17 |             ll x = (ri + k < n) ?
   | rk[ri + k] : -1;
18 |             ll y = (li + k < n) ?
   | rk[li + k] : -1;
19 |             return x < y;
20 |         };
21 |
22 |         sort(sa.begin(), sa.end(),
   | cmp);
23 |
24 |         tmp[sa[0]] = 0;
25 |         for (ll i = 1; i < n; ++i)
   | {
26 |             tmp[sa[i]] = tmp[sa[i -
   | 1]] + cmp(sa[i -
   | 1], sa[i]);
27 |         }
28 |         rk = tmp;
29 |     }
30 |     return sa;
31 | }
32 |
33 | int main() {
34 |     ios::sync_with_stdio(false);
35 |     cin.tie(nullptr);
36 |
37 |     string s; cin >> s;
38 |     vector<ll> sa = suffix_array(s)
   | ;
39 |
40 |     ll n = s.size();
41 |
42 |     ll k; cin >> k;

```

```

43 |     for (ll i = 0; i < k; ++i) {
44 |         string tar; cin >> tar;
45 |         ll m = tar.size();
46 |
47 |         ll l = 0, r = n - 1;
48 |         while (l <= r) {
49 |             ll mid = l + (r - l)/2;
50 |             if (s.substr(sa[mid], m
   | ) < tar) {
51 |                 l = mid + 1;
52 |             } else {
53 |                 r = mid - 1;
54 |             }
55 |         }
56 |         cout << (s.substr(sa[l], m)
   | == tar ? "YES" : "NO"
   | ) << '\n';
57 |     }
58 |
59 |     return 0;
60 | }
61 |

```

10 Tree

10.1 Binary Lifting

```

1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | using ll = long long;
4 |
5 | inline ll flg(ll x) {
6 |     return 63 - __builtin_clzll(x);
7 | }
8 |
9 | inline bool isOnBit(ll x, ll i) {
10 |     return ((1LL << i) & x) > 0;
11 | }
12 |
13 | ll n, q, lgn;
14 | vector<vector<ll>> blf;
15 |
16 | void init() {
17 |     blf[0][1] = -1;
18 |     for (ll u = 2; u <= n; ++u) cin
   | >> blf[0][u];
19 |
20 |     for (ll h = 1; h <= lgn; ++h) {
21 |         for (ll u = 1; u <= n; ++u)
   | {
22 |             ll nt = blf[h - 1][u];
23 |             blf[h][u] = nt == -1 ?
   | -1 : blf[h - 1][nt
   | ];
24 |         }
25 |     }
26 | }
27 |
28 | ll query(ll u, ll step) {
29 |     ll cur = u;
30 |     for (ll i = 30; i >= 0; --i) {
31 |         if (isOnBit(step, i)) {

```

```

31 |             cur = blf[i][cur];
32 |             if (cur == -1) return
   | -1;
33 |         }
34 |     }
35 |     return cur;
36 | }
37 |
38 | int main() {
39 |     ios::sync_with_stdio(false);
40 |     cin.tie(nullptr);
41 |
42 |     cin >> n >> q;
43 |     lgn = flg(n);
44 |     blf.resize(lgn + 1, vector<ll>(
   | n + 1));
45 |     init();
46 |
47 |     while (q--) {
48 |         ll u, step; cin >> u >>
   | step;
49 |         cout << query(u, step) << '
   | \n';
50 |     }
51 |
52 |     return 0;
53 | }

```

10.2 LCA

```

1 | // Use binary lifting
2 |
3 | #include <bits/stdc++.h>
4 | using namespace std;
5 | using ll = long long;
6 | #define pb push_back
7 |
8 | inline ll flg(ll x) {
9 |     return 63 - __builtin_clzll(x);
10 | }
11 |
12 | inline bool isOnBit(ll x, ll i) {
13 |     return ((1LL << i) & x) > 0;
14 | }
15 |
16 | ll n, q, lgn;
17 | vector<ll> d;
18 | vector<vector<ll>> blf, adj;
19 |
20 | void init() {
21 |     blf[0][1] = -1;
22 |     for (ll u = 2; u <= n; ++u) {
23 |         ll v; cin >> v;
24 |         blf[0][u] = v;
25 |         adj[v].pb(u);
26 |     }
27 |
28 |     for (ll h = 1; h <= lgn; ++h) {
29 |         for (ll u = 1; u <= n; ++u)
   | {
30 |             ll nt = blf[h - 1][u];
31 |             blf[h][u] = nt == -1 ?
   | -1 : blf[h - 1][nt

```

```

32 |         }
33 |     }
34 | }
35 | ll query(ll u, ll step) {
36 |     ll cur = u;
37 |     for (ll i = 30; i >= 0; --i) {
38 |         if (isOnBit(step, i)) {
39 |             cur = blf[i][cur];
40 |             if (cur == -1) return
   | -1;
41 |         }
42 |     }
43 |     return cur;
44 | }
45 |
46 | void dfs(ll u, ll dn) {
47 |     d[u] = dn;
48 |     for (ll v : adj[u]) {
49 |         dfs(v, dn + 1);
50 |     }
51 | }
52 |
53 | ll lca(ll u, ll v) {
54 |     if (d[u] > d[v]) swap(u, v);
55 |     if (d[u] < d[v]) v = query(v, d
   | [v] - d[u]);
56 |     if (u == v) return u;
57 |     for (ll h = lgn; h >= 0; --h) {
58 |         ll ntu = blf[h][u];
59 |         ll ntv = blf[h][v];
60 |         if (ntu == -1 or ntv == -1
   | or ntu == ntv)
   |             continue;
61 |         u = ntu;
62 |         v = ntv;
63 |     }
64 |     return blf[0][u];
65 | }
66 |
67 | int main() {
68 |     ios::sync_with_stdio(false);
69 |     cin.tie(nullptr);
70 |
71 |     cin >> n >> q;
72 |     lgn = flg(n);
73 |     d.resize(n + 1);
74 |     blf.resize(lgn + 1, vector<ll>(
   | n + 1));
75 |     adj.resize(n + 1);
76 |     init();
77 |     dfs(1, 0);
78 |
79 |     while (q--) {
80 |         ll u, v; cin >> u >> v;
81 |         cout << lca(u, v) << '\n';
82 |     }
83 |
84 |     return 0;
85 | }

```


ACM ICPC Team Reference - c0mpile_error

Contents

1 Bitwise Trick	1	2 DP	1	5 Geometry	5	6.10 Tarjan 2-SAT	7	8 Others	10
1.1 tricks	1	2.1 LIS	1	5.1 Line Segment Inter- section Test	5	6.11 Topo-sort DFS	7	8.1 mt19937 Usage	10
		2.2 Knapsack	1	5.2 Simple Polygon Area	5	6.12 Dinic 無權二分圖 最大匹配	7	8.2 GCC Builtin Func- tions	10
		2.3 Subset DP	1	5.3 Vector	5	6.13 Dinic Min Cut	8	8.3 distinct k number . .	10
		2.4 Interval DP	1	5.4 Convex Hull	5	6.14 Floyd Warshall . . .	8	8.4 Custom Hash	10
		2.5 LCS	1	6 Graph	5	6.15 MST Kruskal	8	9 String	11
		2.6 Digit DP	1	6.1 Tarjan	5	6.16 Tarjan 縮點	8	9.1 Z-value	11
		3 D&C	2	6.2 Bridge	6	6.17 Kosaraju	9	9.2 KMP	11
		3.1 MergeSort Finds the Number of In- versions	2	6.3 Bellman Ford De- tects Negative Cycle	6	7 Math	9	9.3 Trie	11
		4 Data Structure	2	6.4 Topo-sort Kahn . . .	6	7.1 Fast Exponentiation	9	9.4 Manacher	12
		4.1 Segment Tree	2	6.5 AP	6	7.2 Combinations	9	9.5 String Hashing . . .	12
		4.2 BIT	3	6.6 Dijkstra	6	7.3 Matrix	9	9.6 Suffix Array n log squared n version . .	13
		4.3 Treap	3	6.7 Contracting Ver- tices Based on Spe- cific Conditions . . .	6	7.4 Modular Inverse . .	10	10 Tree	13
		4.4 Sparse Table	5	6.8 MST Prim	6	7.5 Big Integer Addi- tion and Multiplica- tion	10	10.1 Binary Lifting	13
		4.5 DSU	5	6.9 Dinic	7	7.6 Prime Sieve	10	10.2 LCA	13

ACM ICPC Judge Test - c0mpile_error

C++ Resource Test

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 namespace system_test {
```

```
5
6 const size_t KB = 1024;
7 const size_t MB = KB * 1024;
8 const size_t GB = MB * 1024;
9
10 size_t block_size, bound;
11 void stack_size_dfs(size_t depth =
12     1) {
13     if (depth >= bound)
14         return;
15     int8_t ptr[block_size]; // 若無法
16         編譯將 block_size 改成常數
17     memset(ptr, 'a', block_size);
18     cout << depth << endl;
19     stack_size_dfs(depth + 1);
20 }
21 void stack_size_and_runtime_error(
22     size_t block_size, size_t
23     bound = 1024) {
24     system_test::block_size =
25         block_size;
26     system_test::bound = bound;
27     stack_size_dfs();
28 }
```

```
24 }
25
26 double speed(int iter_num) {
27     const int block_size = 1024;
28     volatile int A[block_size];
29     auto begin = chrono::
30         high_resolution_clock::now()
31         ;
32     while (iter_num--)
33         for (int j = 0; j < block_size;
34             ++j)
35             A[j] += j;
36     auto end = chrono::
37         high_resolution_clock::now()
38         ;
39     chrono::duration<double> diff =
40         end - begin;
41     return diff.count();
42 }
```

```
38 void runtime_error_1() {
39     // Segmentation fault
40     int *ptr = nullptr;
41     *(ptr + 7122) = 7122;
42 }
```

```
43
44 void runtime_error_2() {
45     // Segmentation fault
46     int *ptr = (int *)memset;
47     *ptr = 7122;
48 }
49
50 void runtime_error_3() {
51     // munmap_chunk(): invalid
52     // pointer
53     int *ptr = (int *)memset;
54     delete ptr;
55 }
56
57 void runtime_error_4() {
58     // free(): invalid pointer
59     int *ptr = new int[7122];
60     ptr += 1;
61     delete[] ptr;
62 }
63
64 void runtime_error_5() {
65     // maybe illegal instruction
66     int a = 7122, b = 0;
67     cout << (a / b) << endl;
```

```
67 }
68
69 void runtime_error_6() {
70     // floating point exception
71     volatile int a = 7122, b = 0;
72     cout << (a / b) << endl;
73 }
74
75 void runtime_error_7() {
76     // call to abort.
77     assert(false);
78 }
79
80 // namespace system_test
81
82 #include <sys/resource.h>
83 void print_stack_limit() { // only
84     work in Linux
85     struct rlimit l;
86     getrlimit(RLIMIT_STACK, &l);
87     cout << "stack_size = " << l.
88         rlim_cur << " byte" << endl;
```